

```

        fmin=f1;
    } else {
        xmin=x2;
        fmin=f2;
    }
    return xmin;
}
};

```

10.3 Parabolic Interpolation and Brent's Method in One Dimension

We already tipped our hand about the desirability of parabolic interpolation in the bracket routine of §10.1, but it is now time to be more explicit. A golden section search is designed to handle, in effect, the worst possible case of function minimization, with the uncooperative minimum hunted down and cornered like a scared rabbit. But why assume the worst? If the function is nicely parabolic near to the minimum — surely the generic case for sufficiently smooth functions — then the parabola fitted through any three points ought to take us in a single leap to the minimum, or at least very near to it (see Figure 10.3.1). Since we want to find an abscissa rather than an ordinate, the procedure is technically called *inverse parabolic interpolation*.

The formula for the abscissa x that is the minimum of a parabola through three points $f(a)$, $f(b)$, and $f(c)$ is

$$x = b - \frac{1}{2} \frac{(b-a)^2[f(b) - f(c)] - (b-c)^2[f(b) - f(a)]}{(b-a)[f(b) - f(c)] - (b-c)[f(b) - f(a)]} \quad (10.3.1)$$

as you can easily derive. This formula fails only if the three points are collinear, in which case the denominator is zero (the minimum of the parabola is infinitely far away). Note, however, that (10.3.1) is as happy jumping to a parabolic maximum as to a minimum. No minimization scheme that depends solely on (10.3.1) is likely to succeed in practice.

The exacting task is to invent a scheme that relies on a sure-but-slow technique, like golden section search, when the function is not cooperative, but that switches over to (10.3.1) when the function allows. The task is nontrivial for several reasons, including these: (i) The housekeeping needed to avoid unnecessary function evaluations in switching between the two methods can be complicated. (ii) Careful attention must be given to the “endgame,” where the function is being evaluated very near to the roundoff limit of equation (10.2.2). (iii) The scheme for detecting a cooperative versus noncooperative function must be very robust.

Brent's method [1] is up to the task in all particulars. At any particular stage, it is keeping track of six function points (not necessarily all distinct), a , b , u , v , w and x , defined as follows: The minimum is bracketed between a and b ; x is the point with the very least function value found so far (or the most recent one in case of a tie); w is the point with the second least function value; v is the previous value of w ; and u is the point at which the function was evaluated most recently. Also appearing in the algorithm is the point x_m , the midpoint between a and b ; however, the function is not evaluated there.

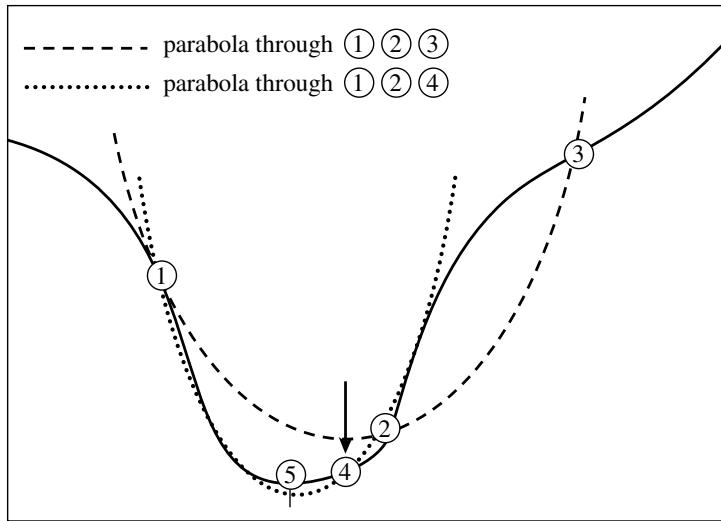


Figure 10.3.1. Convergence to a minimum by inverse parabolic interpolation. A parabola (dashed line) is drawn through the three original points 1,2,3 on the given function (solid line). The function is evaluated at the parabola's minimum, 4, which replaces point 3. A new parabola (dotted line) is drawn through points 1,4,2. The minimum of this parabola is at 5, which is close to the minimum of the function.

You can read the code below to understand the method's logical organization. Mention of a few general principles here may, however, be helpful: Parabolic interpolation is attempted, fitting through the points x , v , and w . To be acceptable, the parabolic step must (i) fall within the bounding interval (a, b) , and (ii) imply a movement from the best current value x that is *less* than half the movement of the *step before last*. This second criterion ensures that the parabolic steps are actually converging to something, rather than, say, bouncing around in some nonconvergent limit cycle. In the worst possible case, where the parabolic steps are acceptable but useless, the method will approximately alternate between parabolic steps and golden sections, converging in due course by virtue of the latter. The reason for comparing to the step *before last* seems essentially heuristic: Experience shows that it is better not to “punish” the algorithm for a single bad step if it can make it up on the next one.

Another principle exemplified in the code is never to evaluate the function less than a distance `tol` from a point already evaluated (or from a known bracketing point). The reason is that, as we saw in equation (10.2.2), there is simply no information content in doing so: The function will differ from the value already evaluated only by an amount of order the roundoff error. Therefore, in the code below you will find several tests and modifications of a potential new point, imposing this restriction. This restriction also interacts subtly with the test for “doneness,” which the method takes into account.

A typical ending configuration for Brent's method is that a and b are $2 \times x \times \text{tol}$ apart, with x (the best abscissa) at the midpoint of a and b , and therefore fractionally accurate to $\pm \text{tol}$.

The calling sequence for `Brent` is exactly analogous to that of `Golden` in the previous section. Indulge us a final reminder that `tol` should generally be no smaller than the square root of your machine's floating-point precision.

mins.h

```
struct Brent : Bracketmethod {
```

Brent's method to find a minimum.

```
    Doub xmin,fmin;
    const Doub tol;
    Brent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &func)
```

Given a function or functor *f*, and given a bracketing triplet of abscissas *ax*, *bx*, *cx* (such that *bx* is between *ax* and *cx*, and *f*(*bx*) is less than both *f*(*ax*) and *f*(*cx*)), this routine isolates the minimum to a fractional precision of about *tol* using Brent's method. The abscissa of the minimum is returned as *xmin*, and the function value at the minimum is returned as *min*, the returned function value.

```
{
```

```
    const Int ITMAX=100;
    const Doub CGOLD=0.3819660;
    const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
```

Here ITMAX is the maximum allowed number of iterations; CGOLD is the golden ratio; and ZEPS is a small number that protects against trying to achieve fractional accuracy for a minimum that happens to be exactly zero.

```
    Doub a,b,d=0.0,etemp,fu,fv,fw,fx;
    Doub p,q,r,tol1,tol2,u,v,w,x,xm;
    Doub e=0.0;
```

This will be the distance moved on the step before last.

```
    a=(ax < cx ? ax : cx);
    b=(ax > cx ? ax : cx);
    x=w=v=bx;
    fw=fv=fx=func(x);
```

a and *b* must be in ascending order, but input abscissas need not be. Initializations...

```
    for (Int iter=0;iter<ITMAX;iter++) {
```

Main program loop.

```
        xm=0.5*(a+b);
        tol2=2.0*(tol1=tol*abs(x)+ZEPS);
        if (abs(x-xm) <= (tol2-0.5*(b-a))) {
```

Test for done here.

```
            fmin=fx;
            return xmin=x;
        }
        if (abs(e) > tol1) {
            r=(x-w)*(fx-fv);
            q=(x-v)*(fx-fw);
            p=(x-v)*q-(x-w)*r;
            q=2.0*(q-r);
            if (q > 0.0) p = -p;
            q=abs(q);
            etemp=e;
            e=d;
```

Construct a trial parabolic fit.

```
            if (abs(p) >= abs(0.5*q*etemp) || p <= q*(a-x)
                || p >= q*(b-x))
                d=CGOLD*(e=(x >= xm ? a-x : b-x));
```

The above conditions determine the acceptability of the parabolic fit. Here we take the golden section step into the larger of the two segments.

```
            else {
                d=p/q;
                u=x+d;
                if (u-a < tol2 || b-u < tol2)
                    d=SIGN(tol1,xm-x);
            }
```

Take the parabolic step.

```
        } else {
            d=CGOLD*(e=(x >= xm ? a-x : b-x));
        }
        u=(abs(d) >= tol1 ? x+d : x+SIGN(tol1,d));
        fu=func(u);
```

This is the one function evaluation per iteration.

```
        if (fu <= fx) {
            if (u >= x) a=x; else b=x;
            shft3(v,w,x,u);
            shft3(fv,fw,fx,fu);
```

Now decide what to do with our function evaluation.

Housekeeping follows: